

TP — Application CodeIgniter 4 : Gestion de Bibliothèque

Partie 1

1.1 Créer une base de données MySQL nommée bibliotheque en UTF-8.

- ☒ Création de la base bibliotheque `app/Database/Schema.sql` en UTF-8

1.2 Créer une table livres contenant les champs suivants : identifiant auto-incrémenté, titre, auteur, ISBN (unique), année de publication, catégorie, résumé, nom de fichier de couverture, statut (disponible / prêté), et les timestamps de création/modification.

- ☒ Création de la table livres

1.3 Créer une table emprunts permettant de tracer chaque prêt : identifiant, référence au livre, nom de l'emprunteur, date d'emprunt et date de retour (nullable).

- ☒ Création de la table emprunt

1.4 Configurer la connexion à la base dans le fichier approprié de CodeIgniter.

- ☒ Configuration de la connexion à la base dans `app/Config/Database.php`

Partie 2

Définir l'ensemble des routes nécessaires à l'application.

2.1 Une route pour afficher la liste de tous les livres (page d'accueil).

- ☒ Création de la route `/` pour l'accueil et pour lister les livres

2.2 Une route pour afficher la fiche détaillée d'un livre, en passant son identifiant dans l'URL.

- ☒ Création du groupe de routes `catalogue`
- ☒ Rajout de la route de l'accueil
- ☒ Création de la route pour la fiche détaillée d'un livre par id `details/(:num)`
- ☒ Rajout au groupe de routes de : `catalogue`

2.3 Deux routes pour l'ajout d'un livre : une pour afficher le formulaire (GET), une pour traiter l'envoi (POST).

- ☒ Création des routes pour l'affichage du formulaire et du traitement de l'ajout d'un livre, route: `ajouter` et suppose l'utilisation de `ajouter` et de `enregistrer`
- ☒ Rajout au groupe de routes de : `catalogue`

2.4 Une route POST pour supprimer un livre.

- ☒ Création de la route pour la suppression de livre par id : `supprimer/(:num)`
- ☒ Rajout au groupe de routes de : `catalogue`

2.5 Deux routes POST pour les actions de prêt et de retour d'un livre, avec l'identifiant du livre en paramètre d'URL.

- ☒ Création du groupe de route `mouvements`
- ☒ Création des routes pour les actions de prêt et de retour d'un livre par id: `preter/( :num)` et `retourner/( :num)`
- ☒ Rajout au groupe de routes de `mouvements`

Justifier le choix de la méthode HTTP pour chaque route liée à une action de modification.

- `POST` pour l'envoi et modification, `GET` pour la lecture

Partie 3

Créer le modèle correspondant à la table livres.

3.1 Déclarer la table, la clé primaire, les champs autorisés et activer les timestamps automatiques.

- ☒ Création du fichier `LivreModel.php` dans `app/Models`
- ☒ Déclaration de la table et de la clé primaire
- ☒ Déclaration des `$allowedFields` et `$useTimestamps`

3.2 Définir les règles de validation suivantes directement dans le modèle :

- Le titre est obligatoire et doit comporter au moins 3 caractères.
- L'auteur est obligatoire.
- L'ISBN est obligatoire et doit être unique en base de données.
- L'année est obligatoire. Rédiger des messages d'erreur explicites en français pour chaque règle.
- ☒ Déclaration de `$validationRules` pour contenir les règles de validations
- ☒ Déclaration de `$validationMessages` pour contenir les retours de messages d'erreurs

3.3 Ajouter une méthode de validation métier pour s'assurer que l'année de publication n'est pas dans le futur.

- ☒ Création de la fonction `anneePublicationValide(int $anneePublication): bool` pour vérifier la date
 - ☒ Prendre l'année courante et comparer les deux variables

3.4 Ajouter une méthode de recherche permettant de filtrer les livres par mot-clé (sur le titre) et optionnellement par catégorie.

- ☒ Création de la fonction `rechercherLivres(?string $motCle = null, ?string $categorie = null): array` pour filtrer les livres par mots-clés
 - ☒ Instancier l'outil `Query Builder`
 - ☒ Vérifier que `$motCle` n'est pas vide et utiliser `like` et `where` pour avoir la réponse filtrée
 - ☒ Retourner le résultat et mettre en ordre avec `orderBy` et `getResultArray()`

3.5 Ajouter une méthode retournant les livres paginés (10 résultats par page).

- ☒ Création de la fonction `getLivresPage(int $parPage = 10, string $group = 'livres')` pour paginer de 10 par page

- ☒ Retourner le résultat d'un `orderBy` et utiliser la fonction `paginate` du framework

Partie 4

Créer le modèle correspondant à la table emprunts.

4.1 Déclarer la table et les champs autorisés.

- ☒ Création du fichier `EmpruntModel.php` dans `app/Models`
- ☒ Déclaration de la table et de la clé
- ☒ Déclaration des `$allowedFields`
- ☒ Déclaration de `$validationRules` pour contenir les règles de validations
- ☒ Déclaration de `$validationMessages` pour contenir les retours de messages d'erreurs

4.2 Ajouter une méthode retournant le dernier emprunt enregistré pour un livre donné (le plus récent selon la date d'emprunt).

- ☒ Création de la fonction `getDernierEmpruntParLivre(int $livreId): ?array`
 - ☒ Retourner le résultat de l'id, utiliser `orderBy` et la fonction `first()`

Partie 5

Créer le contrôleur principal de gestion des livres.

5.1 — Liste (index) : Récupérer les livres avec pagination. Si des paramètres de recherche sont présents dans l'URL (mot-clé ou catégorie), utiliser la méthode de recherche à la place. Transmettre les données à la vue correspondante.

- ☒ Création du fichier `Catalogue.php` dans `app/Controllers`
- ☒ Mise en place des catégories statiques
- ☒ Création de la fonction `index()`
 - ☒ Instanciation des modèles à utiliser
 - ☒ Garder les données du `motCle` et de `categorie`
 - ☒ Vérifier si `motCle` et `categorie` sont différents de vide, faire une recherche si oui
 - ☒ Sinon paginer la page
 - ☒ Boucle foreach par référence à `$livres`, pour deviner et avoir le statut d'un livre
 - ☒ Mise en place dans le tableau `$data` et retour dans la vue

5.2 — Détail : Récupérer un livre par son identifiant. Si le livre n'existe pas, déclencher une page 404. Récupérer également le dernier emprunteur via `EmpruntModel` et transmettre les deux à la vue.

- ☒ Création de la fonction `details(int $id)` dans `Catalogue.php`
 - ☒ Instancier `LivreModel` et utiliser `find($id)` pour trouver le livre
 - ☒ Vérifier si le livre existe, sinon lever une erreur 404 (`PageNotFoundException`)
 - ☒ Instancier `EmpruntModel` et récupérer le dernier emprunt via `getDernierEmpruntParLivre($id)`
 - ☒ Passer les données (`livre` et `dernier_emprunt`) à la vue `details`

5.3 — Formulaire : Afficher simplement le formulaire d'ajout.

- ☒ Création de la fonction `ajouter()` dans `Catalogue.php`

- ☒ Récupérer et transmettre les catégories statiques à la vue
- ☒ Afficher la vue `ajouter`

5.4 — Enregistrement (store) : Récupérer les données POST. Valider l'année manuellement (règle métier). Si une image de couverture est envoyée, vérifier que c'est bien un fichier image (jpeg, png ou webp) et que sa taille ne dépasse pas 2 Mo. En cas d'erreur, réafficher le formulaire avec un message. Sinon, enregistrer le fichier dans `public/uploads/`. Utiliser le modèle pour insérer les données ; en cas d'échec de validation, réafficher le formulaire avec les erreurs. En cas de succès, rediriger vers la liste avec un message flash de confirmation.

- ☒ Création de la fonction `enregistrer()` dans `Catalogue.php`
 - ☒ Récupérer les données envoyées avec `$this->request->getPost()` (titre, auteur, etc.)
 - ☒ Appliquer la vérification de l'année via `anneePublicationValide` du modèle, sinon rediriger (`withInput()`) avec l'erreur
 - ☒ Préparer le tableau de données `$data_livre` avec statut par défaut sur "disponible"
 - ☒ Gérer l'upload d'image : récupérer le fichier avec `$this->request->getFile('couverture')`
 - ☒ Utiliser le validateur natif de CI4 pour confirmer que c'est une image (png/jpeg/webp) de max 2Mo
 - ☒ Déplacer l'image vers `FCPATH . 'uploads/'` avec un nom aléatoire (`getRandomName()`)
 - ☒ Instancier `LivreModel` et utiliser `insert($data_livre)`
 - ☒ Si erreur du modèle, retourner au formulaire avec les erreurs associées (`withInput() ->with('errors', ...)`)
 - ☒ Si insertion réussie, redirection vers la page d'accueil avec message de réussite

5.5 — Suppression : Supprimer le livre et rediriger avec un message flash.

- ☒ Création de la fonction `supprimer($id)` dans `Catalogue.php`
 - ☒ Vérifier l'existence du livre avec `find($id)`
 - ☒ Supprimer le livre avec `delete($id)`
 - ☒ Redirection vers `/catalogue` avec un message flash de succès

Partie 6

Créer le contrôleur gérant les mouvements de livres.

6.1 — Prêt : Vérifier que le livre existe et qu'il est disponible. Si ce n'est pas le cas, rediriger avec un message d'erreur. Sinon, enregistrer un nouvel emprunt avec le nom de l'emprunteur et la date du jour, puis mettre à jour le statut du livre à "prêté".

- ☒ Création du fichier `Mouvements.php` dans `app/Controllers`
- ☒ Création de la méthode `preter(int $id)`
 - ☒ Instancier `LivreModel` -> récupération du livre via `find($id)`
 - ☒ Vérifier s'il est disponible, sinon rediriger avec une erreur
 - ☒ Instancier `EmpruntModel` et stocker les données (`emprunteur`, `livre_id`, `date_emprunt`)
 - ☒ Mettre à jour le livre sur l'état "prêté" par méthode `update($id, data)`
 - ☒ Redirection avec un message flash de réussite

6.2 — Retour : Retrouver l'emprunt actif (sans date de retour) pour ce livre, renseigner la date de retour, et remettre le statut du livre à "disponible".

- ☒ Création de la méthode `retourner(int $id)` dans `Mouvements.php`
 - ☒ Instancier `EmpruntModel` et utiliser la méthode appropriée pour récupérer l'emprunt actif sans date de fin (`where date_retour IS NULL`)
 - ☒ Mise à jour de la `date_retour = date('Y-m-d')` (aujourd'hui) et enregistrement
 - ☒ Mise à jour du livre lié pour repasser l'état à "disponible" (via `LivreModel`)
 - ☒ Rediriger vers l'accueil avec un message de succès

Partie 7

Créer l'ensemble des vues en utilisant le système de templating de CodeIgniter (layout + sections).

7.1 — Layout principal : Créer un fichier de mise en page commune incluant la barre de navigation, la zone d'affichage des messages flash (succès et erreur), et la zone de contenu dynamique.

- ☒ Création du fichier `layout.php` (ou `default.php`) dans `app/Views/`
 - ☒ Structure HTML de base (doctype, head, body)
 - ☒ Placer la barre de navigation
 - ☒ Récupérer les données flash (s'il y en a) et les afficher : `session()->getFlashdata('success')`, etc.
 - ☒ Ajouter le bloc principal dynamique : `<?= $this->renderSection('content') ?>`

7.2 — Vue catalogue : Afficher un formulaire de recherche (champ texte + liste de catégories). Afficher un tableau listant tous les livres avec : titre (lien vers la fiche), auteur, année, statut coloré (vert = disponible, rouge = prêté). Pour chaque livre disponible : afficher un formulaire inline pour saisir le nom de l'emprunteur et un bouton "Prêter". Pour chaque livre prêté : afficher uniquement le bouton "Retourner". Afficher un bouton "Supprimer" avec confirmation JavaScript. Afficher les liens de pagination si nécessaire.

- ☒ Création du fichier `index.php` dans `app/Views/catalogue/` (ou `app/Views/`)
 - ☒ Extension du template de base `layout.php` via `extend()` et `section('content')`
 - ☒ Ajout du formulaire (`<form>`) de recherche en GET vers la page d'accueil (/)
 - ☒ Boucle `foreach` sur le tableau `$livres`, affiché sous forme de tableau HTML (`<table>`)
 - ☒ Logique d'affichage conditionnelle (if/else) selon le statut du `$livre` (Afficher formulaire "Prêter" (POST) vs bouton "Retourner" (POST))
 - ☒ Ajout du bouton (POST) de suppression dans un `<form>` avec confirmation JavaScript : `onsubmit="return confirm('Confirmer ?')"`
 - ☒ Affichage des liens de pagination : `<?= $pager->links('livres') ?>`

7.3 — Vue détail : Afficher toutes les informations du livre ainsi que le nom et la date d'emprunt du dernier emprunteur (si disponible).

- ☒ Création du fichier `details.php` dans `app/Views/` (ou `catalogue/`)
 - ☒ Affichage de toutes les données de `$livre`
 - ☒ Si le tableau `$dernier_emprunt` a été passé dynamiquement : affichage de la date et du nom de l'emprunteur

7.4 — Formulaire d'ajout : Champs : titre, auteur, ISBN, année, catégorie (liste déroulante), résumé, couverture (upload). Afficher les messages d'erreur de validation sous chaque champ concerné. Utiliser `old()`

pour repopuler les champs en cas d'erreur. Inclure le jeton CSRF. Limiter l'année maximale à l'année en cours côté client.

- ☒ Création de la page `ajouter.php` avec layout
 - ☒ Création du `<form>` avec méthode `POST` et `enctype="multipart/form-data"` pointant sur l'action `enregistrer`
 - ☒ Ajout du jeton CSRF via `<?= csrf_field() ?>`
 - ☒ Remplissage des anciennes valeurs grâce à `<?= old('titre') ?>`
 - ☒ Affichage conditionnel des erreurs de validation (ex: `$errors['titre']`)
 - ☒ Ajout de la limite max (`max="<?= date('Y') ?>"`) pour l'`<input type="number">` correspondant à l'année

Partie 8

8.1 Activer le filtre CSRF globalement pour toutes les requêtes.

- ☒ Dans `app/Config/Filters.php`, repérer la propriété `$globals`
- ☒ Activer pour une sécurisation complète (`'csrf'` sous `before`)

8.2 S'assurer que tous les formulaires POST contiennent le jeton CSRF.

- ☒ Contrôle et ajout de `<?= csrf_field() ?>` sur tous les formulaires liés (ajout, suppression, prêt, retour)

8.3 Utiliser `esc()` sur toutes les données affichées dans les vues pour prévenir les failles XSS.

- ☒ Partout, dans chaque vue, utiliser `<?= esc($variable) ?>` au lieu d'un appel direct.

Critères d'évaluation

- Fonctionnement correct des opérations CRUD
- Respect de la validation et affichage des erreurs
- Protection CSRF active et fonctionnelle
- Gestion correcte du statut (prêt / retour / désactivation du bouton)
- Pagination visible dès 11 livres
- Validation du fichier image (type + taille)
- Code organisé selon le pattern MVC
- Vues propres, utilisant le layout et `esc()`